PHASE 6 OF 8

API Reference & User Interface

Every endpoint with request and response shapes, ready-to-run curl examples, and a tour of the interface.

Endpoints | curl Examples | UI Tour | Data Flows

6.1 API Endpoints Summary

Method	Path	Request	Response
GET	/	—	Renders the frontend HTML page
GET	/qa	?question=...	{"answer": "..."}
POST	/explain	{"topic": "..."}	{"topic": "...", "explanation": "..."}
POST	/summarize	{"text": "..."}	{"summary": "..."}
POST	/quiz	{"text": "..."}	{"quiz": [{question, options, answer}, ...]}
GET	/learn/recommendations	?topic=...	{"topic": "...", "recommendation": "..."}

Validation behaviour

Every POST route checks that its required field is not empty. If it is, the API returns HTTP **400** with a JSON body such as `{"error": "Please provide a topic."}`.

6.2 Example API Usage

Start the server with `uvicorn main:app --reload`, then try these from a terminal.

```
# Q&A
curl "http://127.0.0.1:8000/qa?question=Why%20is%20the%20sky%20blue%3F"

# Explain
curl -X POST http://127.0.0.1:8000/explain \
  -H "Content-Type: application/json" -d '{"topic": "Photosynthesis"}'

# Summarize
curl -X POST http://127.0.0.1:8000/summarize \
  -H "Content-Type: application/json" \
  -d '{"text": "Paste a long passage here..."}'

# Quiz
curl -X POST http://127.0.0.1:8000/quiz \
  -H "Content-Type: application/json" -d '{"text": "python"}'

# Learning path
curl "http://127.0.0.1:8000/learn/recommendations?topic=Machine%20Learning"
```

6.3 Interface Walkthrough

The screens below are taken from a running instance of EduGenie, showing the landing page and the Quiz feature end to end.

Landing page

The home page presents three of EduGenie's tools — Q&A, Explain, and Summarize — as simple, single-purpose forms.

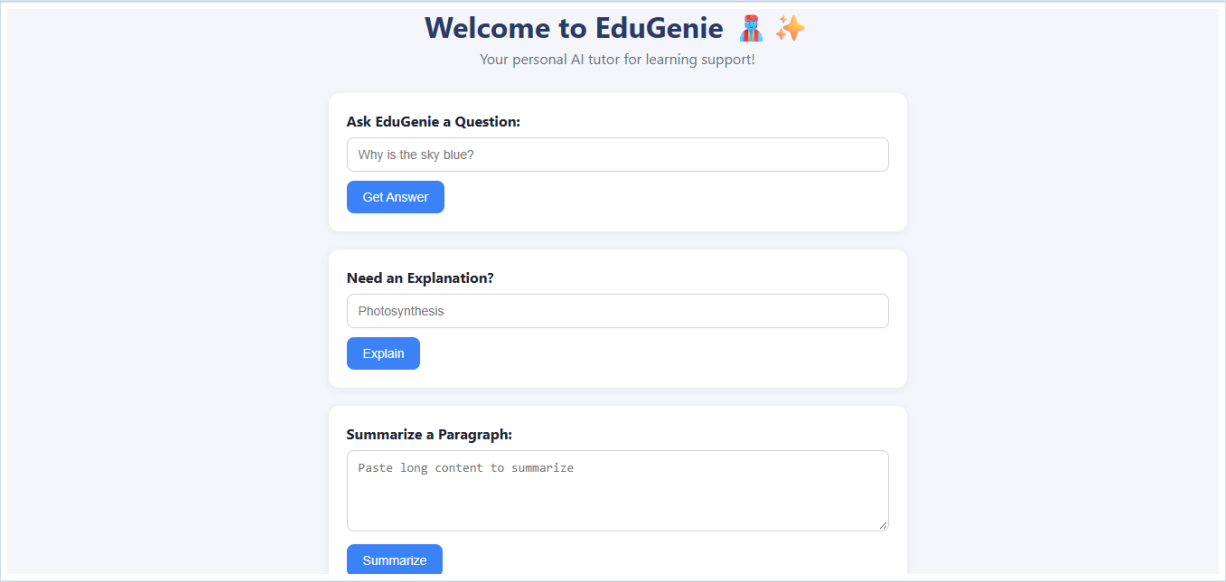


Figure — EduGenie landing page: Ask a Question, Need an Explanation, and Summarize a Paragraph.

Quiz tab — generating questions

A user types a topic (here, "python") and clicks **Generate Quiz**. The backend calls `quiz_module.py`, which returns a structured list of questions.

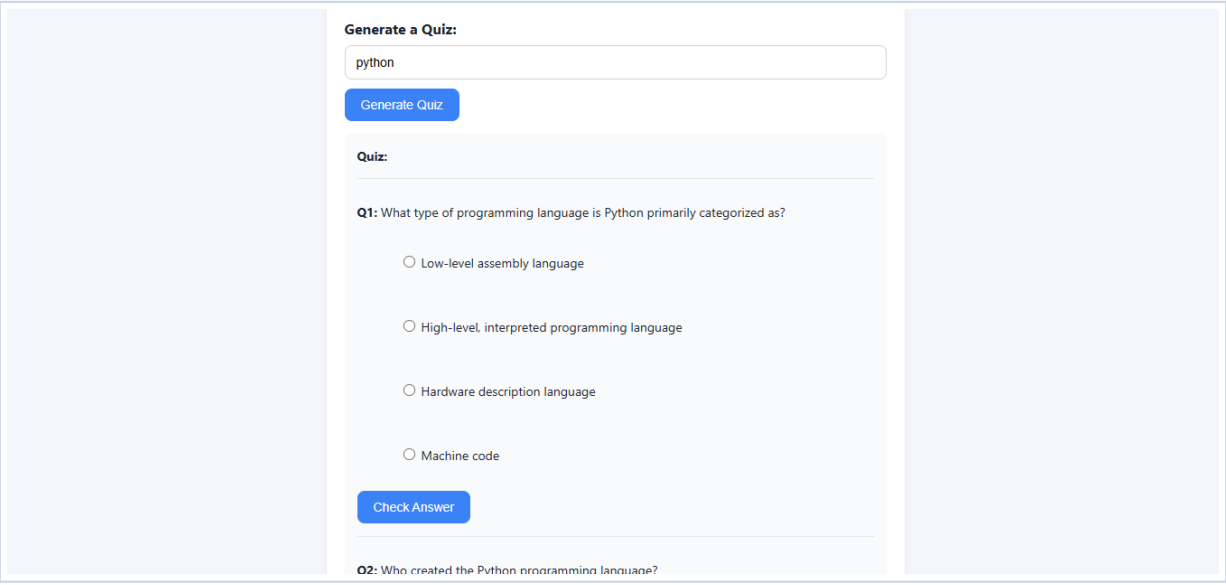


Figure — Quiz tab: three multiple-choice questions generated from the topic "python."

Quiz tab — checking answers

Each question is graded independently when the user clicks **Check Answer**, with immediate correct / incorrect feedback shown beneath the question.

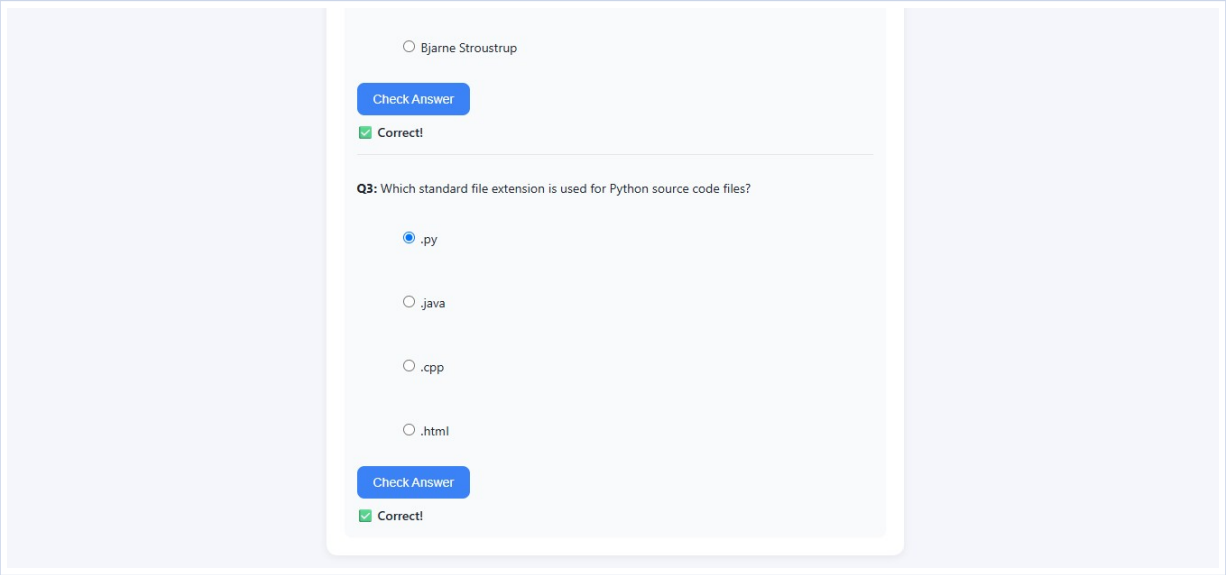


Figure — Quiz tab: the last question answered correctly, matching the auto-grading described in quiz_module.py.

6.4 Feature Data Flows

Each feature follows the same shape — user input, a constructed prompt, a model call and a formatted response.

Feature	Input	Prompt	Model	Output
Q&A	Free-form question	Direct pass-through	Gemini chat	`{"answer"}`
Explain	Topic name	“For a school student”	Local LaMini-Flan-T5	`{"topic", "explanation"}`
Summarize	Pasted passage	“Summarize in simple language”	Gemini chat	`{"summary"}`
Quiz	Passage or topic	3 MCQs as strict JSON	Gemini + JSON cleaner	`{"quiz": [...]}`
Learning path	Topic name	Topics, order, resources, levels	Gemini chat	`{"topic", "recommendation"}`

UI improvement worth making

A loading indicator while a Gemini-backed request is in flight would make the interface feel more responsive, particularly for Quiz and Learning Path, which return the longest responses.